# torch.jit.save#

https://pytorch.org/docs/stable/generated/torch.jit.save.html#torch.jit.save

**Description:**

Save an offline version of this module for use in a separate process. The saved module serializes all of the methods, submodules, parameters, and attributes of this module. It can be loaded into the C++ API using torch::jit::load(filename) or into the Python API with torch.jit.load. To be able to save a module, it must not make any calls to native Python functions. This means that all submodules must be subclasses of ScriptModule as well. All modules, no matter their device, are always loaded onto the CPU during loading. This is different from torch.load()'s semantics and may change in the future.

## Function Signature

```
torch.jit.save(m, f, _extra_files=None)[source]#
Parameters
```

## Code Examples

```
import torch
import io

class MyModule(torch.nn.Module):
    def forward(self, x):
        return x + 10

m = torch.jit.script(MyModule())

# Save to file
torch.jit.save(m, 'scriptmodule.pt')
# This line is equivalent to the previous
m.save("scriptmodule.pt")

# Save to io.BytesIO buffer
buffer = io.BytesIO()
torch.jit.save(m, buffer)

# Save with extra files
extra_files = {'foo.txt': b'bar'}
torch.jit.save(m, 'scriptmodule.pt', _extra_files=extra_files)
```

## Notes & Warnings

NOTE Danger All modules, no matter their device, are always loaded onto the CPU during loading. This is different from torch.load()'s semantics and may change in the future.

NOTE Note torch.jit.save attempts to preserve the behavior of some operators across versions. For example, dividing two integer tensors in PyTorch 1.5 performed floor division, and if the module containing that code is saved in PyTorch 1.5 and loaded in PyTorch 1.6 its division behavior will be preserved. The same module saved in PyTorch 1.6 will fail to load in PyTorch 1.5, however, since the behavior of division changed in 1.6, and 1.5 does not know how to replicate the 1.6 behavior.

# Detailed Documentation

## Documentation

Save an offline version of this module for use in a separate process.

The saved module serializes all of the methods, submodules, parameters, and attributes of this module. It can be loaded into the C++ API using torch::jit::load(filename) or into the Python API with torch.jit.load.

To be able to save a module, it must not make any calls to native Python functions. This means that all submodules must be subclasses of ScriptModule as well.

All modules, no matter their device, are always loaded onto the CPU during loading. This is different from torch.load()’s semantics and may change in the future.

m – A ScriptModule to save.

f – A file-like object (has to implement write and flush) or a string containing a file name.

_extra_files – Map from filename to contents which will be stored as part of f.

torch.jit.save attempts to preserve the behavior of some operators across versions. For example, dividing two integer tensors in PyTorch 1.5 performed floor division, and if the module containing that code is saved in PyTorch 1.5 and loaded in PyTorch 1.6 its division behavior will be preserved. The same module saved in PyTorch 1.6 will fail to load in PyTorch 1.5, however, since the behavior of division changed in 1.6, and 1.5 does not know how to replicate the 1.6 behavior.

Example: .. testcode: